Agentic Real-Time DAST

Security Agents That Hunt Vulnerabilities as the Platform Changes

TBO Security Hackathon 2026

Participant Pack · Problem Statement

Context

B2B travel platforms change all the time. New APIs, booking and payment changes, supplier integrations, and role updates go live every day. Each change can open a vulnerability, and the most damaging ones are often business-logic flaws. One agency reads another's bookings; a fare is changed in the request, a refund is paid twice, or a sub-agent quietly gains admin rights.

Most application security testing is still periodic and generic. Scans run on a schedule, treat every endpoint the same way, miss business-logic abuse, and produce long lists of false positives. By the time a real finding reaches its owner, the code has already moved on.

The Challenge

Design and build autonomous security agents that test a live, production-like travel platform in near real time. They should notice what changed, understand how the application and its business flows work, plan and run targeted attacks safely, prove each finding, and report it to the right owner within minutes of a deploy.

Your agents should help decide:

- What changed since the last run, and what needs testing now
- Which attacks make sense for each endpoint, given its parameters, roles, and business purpose
- Which findings are real and exploitable, and which are noise
- How serious a finding is for this business (bookings, payments, credit, PII, supplier access)
- When to proceed, slow down, stop, or hand over to a human
- As part of building the API inventory, which endpoints are active, which are zombie (still live but unused), and which are shadow (in use but undocumented)

Teams may use any approach: LLM agents, classic scanners, rule engines, fuzzers, or hybrids.

The Agent Loop

Submissions should cover as much of this loop as they can. Teams may go deep on a few stages if they show how the rest would plug in.

Stage	What the agent does
Detect change	Reacts to a deploy, a new or changed endpoint seen in traffic, new front-end JavaScript bundles, or a new subdomain
Learn the app and build inventory	Discovers endpoints and parameters without documentation by crawling the UI with test accounts at each role, recording browser traffic, reading front-end JavaScript for API routes, and probing for hidden or undocumented paths, then builds its own inferred API map. Compares that map against real traffic over a set window, such as 30 or 90 days, and flags endpoints that still respond but get no legitimate traffic (zombie) or that are used but were never mapped (shadow)
Plan attacks	Chooses targeted tests per endpoint instead of spraying every payload

Stage	What the agent does
Execute safely	Sends scoped, rate-limited requests and backs off if the app shows strain
Verify	Confirms each hit with a minimal, clean proof so only real issues survive
Triage	Removes duplicates, scores severity in business context, and maps each finding to an owner
Report	Raises a ticket with evidence, impact, severity, owner, and a suggested fix
Retest	Reruns the proof after a fix and closes the finding once it is resolved

Reference Architecture (suggested, not mandatory)

- **Change triggers:** CI/CD deploy webhooks, API gateway or load balancer traffic logs, front-end bundle diffs, and DNS and certificate watchers
- **Orchestrator agent:** decides what to test, in what order, and pauses everything when the safety layer raises an alarm
- **Recon agent and app knowledge graph:** a stored map of endpoints, parameters, roles, and flows such as search, hold, book, pay, and cancel
- **Inventory agent (zombie / shadow APIs):** compares the knowledge graph against traffic logs from the last 30 to 90 days to label each endpoint active, zombie, or shadow
- **Attack planner:** reasons over the knowledge graph to pick targeted tests
- **Specialist executor agents:** one per vulnerability class, wrapping tools such as ZAP, Nuclei, or sqlmap where useful, with LLM reasoning for business-logic tests
- **Safety layer:** scope allowlist, rate limiting, health monitoring, a test-credential vault, and an audit log of every request
- **Verifier, triage, reporter, and retest agents:** turn raw hits into confirmed, ranked, ticket-ready findings and close the loop after fixes
- **Human reviewer:** approves risky tests and checks high-severity findings before escalation

The diagram below shows how these components fit together.

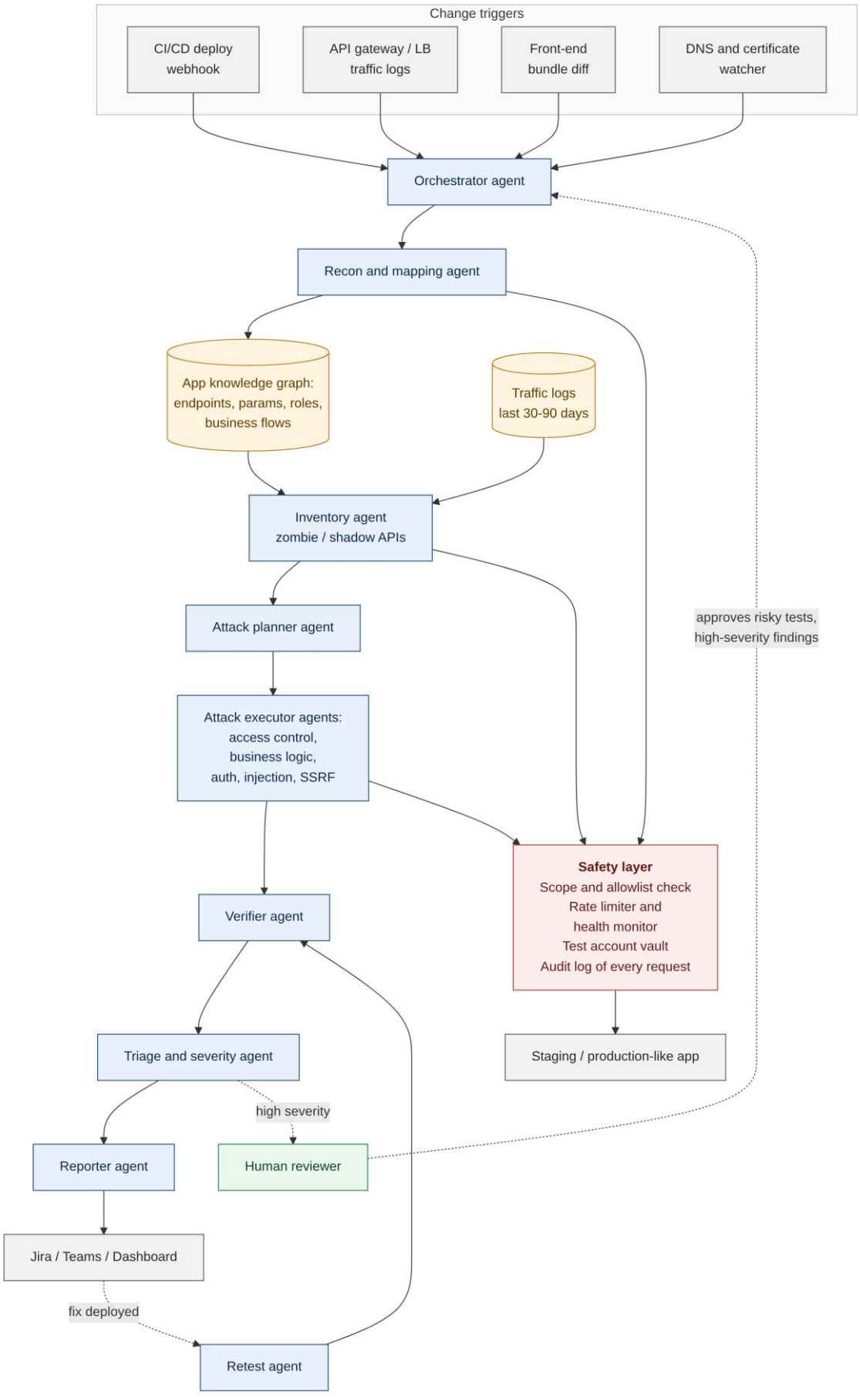


Figure 1. Reference architecture for agentic real-time DAST.

Vulnerability Classes in Scope

Teams may address one class deeply or reason across several.

Class	Examples of risk (illustrative, not exhaustive)
Access control	IDOR on booking, invoice, or traveller IDs, cross-agency data access, sub-agent to admin escalation
Business logic	Fare or markup tampering, coupon or discount reuse, cancel-rebook refund abuse, skipped payment steps, negative quantities
Authentication and sessions	Weak password reset, token reuse, session fixation, JWT flaws, missing step-up auth on sensitive actions
Injection	SQL, NoSQL, command, template, and header injection in search, filters, and supplier callbacks
API hygiene	Unauthenticated new endpoints, mass assignment, excessive data in responses, missing rate limits
Data exposure	Passport, card, or contact data in responses, error messages, logs, or debug endpoints
Server-side risks	SSRF through supplier or webhook URLs, insecure file upload, open redirects
Zombie and shadow APIs	Old API versions (/v1/ still live after /v3/ shipped), retired partner or supplier endpoints, debug and test routes left in production, endpoints from decommissioned mobile app builds, services on forgotten hosts

Additional requirement: zombie and shadow API inventory

Alongside the core objective, agents should flag zombie and shadow endpoints as part of the discovery output. This is a secondary requirement, not the main goal. Useful techniques include guessing older versions and naming patterns from known routes (for example /v1/, /old/, /internal/, /test/, /beta/), reading old front-end JavaScript bundles and mobile app builds for routes no longer called, checking public archives such as the Wayback Machine, reading gateway and load balancer route configs for paths with no matching traffic, and scanning old subdomains and hosts from DNS history and certificate transparency logs.

For each zombie or shadow endpoint the report should note when it last saw real traffic, whether it is reachable without auth, what data it exposes, which vulnerabilities it has compared with its current version, the likely owner, and a recommendation to decommission, restrict, or bring it up to date.

Environment and Inputs You May Assume or Use

Teams get a production-like staging environment of selected travel flows, never real customer systems or data. No OpenAPI or other API documentation is provided; agents must discover the API for themselves. Inputs may include:

- Access to the running web portal and mobile or partner APIs, with no documentation
- Test accounts at several roles across at least two agencies
- Sample recorded traffic (HAR files or proxy logs) from normal user journeys
- API gateway and load balancer access logs for the last 30 to 90 days
- CI/CD deploy events and release notes
- DNS records, certificates, and service fingerprints
- Application and access logs
- Asset ownership data and past findings

Planted vulnerabilities exist, but not all are documented. New changes will be deployed during the event to see whose agents catch them first. Teams may invent more inputs if they explain why they matter.

Rules of Engagement

Every agent must:

- Stay within the approved target list and stop at anything outside it
- Use only the provided test accounts and synthetic data
- Rate-limit itself and never run denial-of-service or destructive tests
- Prove findings with minimum evidence, without bulk data extraction or keeping access
- Log every request so reviewers can replay what the agent did

Breaking these rules disqualifies a submission, however many findings it has.

What a Strong Submission Shows

1. **Speed:** short time from a deploy to a verified finding
2. **Depth:** real, reproducible vulnerabilities, especially business-logic flaws that conventional scanners miss
3. **Precision:** few false positives and duplicates
4. **Context:** severity judged by what the endpoint means for the business, not only a generic score
5. **Safe autonomy:** stays in scope, controls its own load, and knows when a human must decide
6. **Actionability:** each finding is ready to drop into a ticket and fix, and is retested automatically
7. **Discovery and inventory:** builds an accurate map of the API without being given a spec, and labels endpoints as active, zombie, or shadow with a recommended action for each; bonus points if it generates an OpenAPI spec as a by-product

Judging Weightage (suggested)

Criterion	Weight
Valid findings and severity	30%
Precision and noise control	20%
Safety and scope discipline	20%
Speed after change	10%
Report quality and actionability	10%
Discovery and API inventory	10%
Total	100%